

Optimized ATS Engine

Day 18 – Optimization & Performance Tuning

1. Overview

Day 18 focused on optimizing the existing ATS system for improved processing efficiency, stability, resume handling, and semantic matching performance.

Rather than introducing a separate ATS architecture, the optimization work was applied directly to existing system components while preserving previously implemented functionality.

The optimization covered:

- PDF text extraction
- Resume text cleaning
- Resume normalization
- Resume section classification
- Header and profile detection
- Semantic model loading
- Semantic embedding generation
- Memory handling
- Noisy resume processing

2. Optimization Objectives

The primary objectives were to:

- Improve text extraction efficiency.
- Reduce unnecessary string and memory operations.
- Reduce repeated AI model loading.
- Improve semantic matching response time.
- Process semantic inputs in batches.
- Improve noisy resume handling.
- Improve resume header/profile detection.
- Reduce unnecessary UNKNOWN section classification.
- Maintain compatibility with existing ATS functionality.
- Validate system stability after optimization.

3. PDF Text Extraction Optimization

Previous Processing Approach

PDF text extraction can become inefficient when extracted page text is repeatedly appended to a growing string.

Repeated string concatenation may create unnecessary intermediate string objects as the resume grows.

Optimized Approach

The PDF reader now stores extracted page text in a list:

```
Page 1 Text
Page 2 Text
Page 3 Text
    ↓
List of Page Text
    ↓
Single Join Operation
    ↓
Final Resume Text
```

The optimized implementation uses page-wise extraction and combines the results only once.

The PDF document is also handled using a context-managed workflow to ensure that document resources are released after processing.

Benefits

- Reduced repeated string allocation
- Cleaner memory handling
- Efficient multi-page extraction
- Safe empty-page handling
- Proper document resource management

4. Noisy Resume Handling

Resume files may contain inconsistent formatting caused by:

- PDF conversion
- Copy-and-paste operations
- Different bullet characters
- Excessive whitespace
- Decorative separators
- Incorrectly decoded characters
- Irregular line endings

The text-cleaning component was improved to normalize these variations before downstream processing.

5. Unicode Normalization

Unicode normalization was introduced using the NFKC normalization form.

This helps standardize equivalent character representations before further text processing.

The cleaning pipeline now follows:

```
Raw Resume Text
  ↓
Unicode Normalization
  ↓
Line Ending Normalization
  ↓
Control Character Removal
  ↓
Bullet Normalization
  ↓
Separator Reduction
  ↓
Whitespace Normalization
  ↓
Section Heading Normalization
  ↓
Clean Resume Text
```

6. Bullet and Formatting Normalization

Different bullet representations are converted into a consistent format.

Examples include:

-
-
-
-
- ▶
- *

Common incorrectly decoded bullet text is also handled.

This reduces formatting differences that could otherwise interfere with resume parsing and section processing.

7. Compiled Regular Expressions

Frequently used regular expressions are compiled once when the module loads.

These patterns handle:

- Horizontal whitespace
- Blank lines
- Control characters
- Repeated dashes
- Repeated equal signs
- Repeated underscores
- Section-heading normalization

This avoids repeatedly rebuilding identical regular-expression patterns during resume processing.

8. Resume Normalization Optimization

The Resume Normalizer was refined to improve structured-data handling.

The optimized normalizer supports:

- Text normalization
 - List normalization
 - Duplicate removal
 - Nested dictionary handling
 - Personal attribute masking
 - Consistent lowercase keys
 - Safe null-value handling
-

9. Efficient Duplicate Detection

Normalized list values now use a set for duplicate tracking while preserving the original order.

For example:

```
Python
Python
SQL
Python
```

becomes:

```
python
sql
```

This provides cleaner candidate information before downstream evaluation.

10. Profile and Header Detection Improvement

One important optimization involved the Resume Section Classifier.

Previously, information appearing before the first recognized section heading could be classified as:

UNKNOWN

However, the beginning of a resume commonly contains:

- Candidate name
- Current designation
- Professional title
- Contact information
- Profile information

Therefore, this content should not automatically be treated as unknown data.

11. Updated Classification

The classifier now initializes resume content under:

PROFILE

instead of:

UNKNOWN

Example:

John Doe
Data Scientist
john@example.com

SKILLS
Python
SQL

The resulting structure becomes:

PROFILE

John Doe
Data Scientist
john@example.com

SKILLS

Python
SQL

This reduces unnecessary UNKNOWN classifications and improves the representation of resume header information.

12. Additional Section Heading Support

The section classifier was expanded to recognize additional common heading variations.

Examples include:

Profile

Profile
Professional Profile
Summary
Professional Summary
Career Summary
Objective
Career Objective

Skills

Skills
Technical Skills
Core Competencies
Key Skills
Technical Competencies

Experience

Experience
Work Experience
Professional Experience

Employment History
Career History

Education

Education
Academic Qualifications
Qualification
Academic Background
Educational Qualifications

Additional variants were also included for certifications and projects.

13. Semantic Model Loading Optimization

Semantic matching uses the SentenceTransformer model:

```
all-MiniLM-L6-v2
```

Model initialization is relatively expensive compared with ordinary Python operations.

Repeatedly creating independent model instances can therefore introduce unnecessary processing overhead.

14. Shared Model Reuse

The optimized Semantic Matching Engine stores the model at the class level.

Conceptually:

```
First Semantic Engine
    ↓
Load SentenceTransformer
    ↓
Store Shared Model
    ↓
Second Semantic Engine
    ↓
Reuse Existing Model
```

The model is loaded only when no shared model instance is currently available within the process.

Subsequent engine instances reuse the existing model.

15. Batch Semantic Processing

Previously, semantic comparison could require separate encoding operations for each resume and job-description component.

The optimized overall similarity workflow groups:

```
Resume Skills
JD Skills
Resume Experience
JD Experience
Resume Projects
JD Projects
```

into a single batch.

The model generates the six embeddings together.

The resulting embedding pairs are then compared for:

```
Skills Similarity
Experience Similarity
Projects Similarity
```

This reduces the number of separate model encode calls required by the overall semantic comparison workflow.

16. Normalized Embeddings

Embedding generation now uses normalized embeddings.

Because the embeddings are normalized, similarity can be calculated efficiently using a dot product.

The semantic workflow becomes:

```
Text Inputs
↓
Batch Encoding
↓
Normalized Embeddings
↓
Dot Product
↓
Similarity Percentage
```

17. Optimization Validation

The Day 18 optimization test validated:

Noisy resume cleaning	PASSED
Header/Profile detection	PASSED
Resume normalization	PASSED
PDF extraction	PASSED
Semantic model reuse	PASSED
Semantic batch processing	PASSED
Memory stability	PASSED

Final test result:

All Day 18 Optimization and Performance tests passed successfully!

18. Regression Validation

Because semantic matching was optimized, the original semantic matching test was rerun to verify that existing behavior remained functional.

Regression result:

Skills Similarity:	76.63
Experience Similarity:	66.39
Projects Similarity:	84.05
Overall Similarity:	75.69
Match Level:	Good Match

Final regression status:

All Semantic Matching tests passed successfully!

This confirms that the optimized implementation preserved the previously tested semantic matching behavior for the existing test case.

19. Conclusion

Day 18 optimization improved several internal ATS processing components without introducing a separate architecture.

The optimized system now includes more efficient PDF text construction, stronger noisy-text normalization, improved resume normalization, profile/header classification, shared semantic model reuse, batch embedding generation, and normalized similarity calculations.

Performance and regression tests completed successfully, providing a stable optimized baseline for subsequent ATS development.